# 1 MPI Parallisation

The global finite difference grid is decomposed into a user defined Cartesian process grid $P_x \times P_y \times P_z$. Each MPI rank is assigned a single local block and stores only its owned interior data together with an additional layer of halo ghost cells. For non-divisible grid sizes, a load balanced block decomposition method is employed. Each remainder points assigned to the first ranks in each coordinate direction. Figures 1.1 and 1.2 gives an example of of global domain decomposition.

The nature of the problem means that the 7 point finite difference stencil (6 neighbours and 1 central point) only requires the nearest neighbour data. As such, the Cartesian decomposition method was chosen as this is most natural and communication occurs only across shared faces.

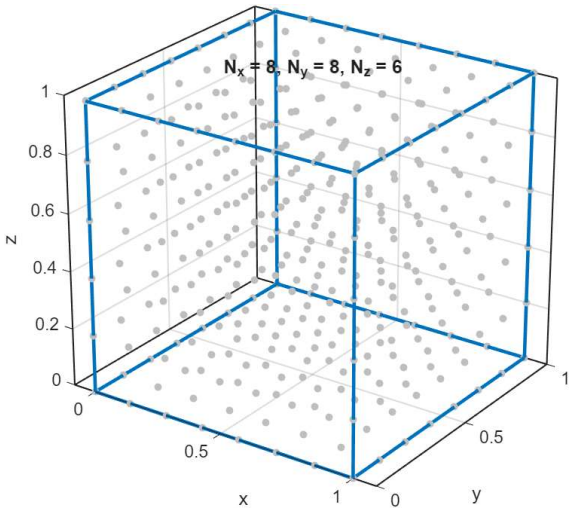


**Figure 1.1:** Global Composition Domain

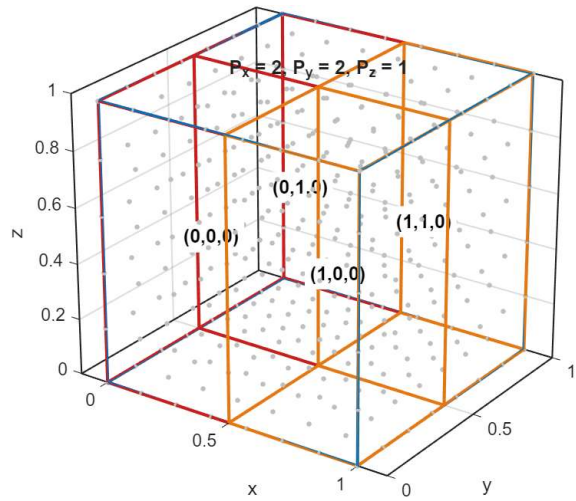


**Figure 1.2:** Cartesian MPI Domain Decomposition

Should the number of grid points not be divisible by the number of ranks in a coordinate direction, an extra point is assigned to the first remainder rank. Figure 1.3 provides an overview with Px = 3 and Global Nx = 10.

Each rank stores its local subdomain in a padded array that includes 1 layer of halo cells on all sides. The owned interior corresponds to indices:

$$i = 1...N_x^{\text{loc}}, j = 1...N_y^{\text{loc}}, k = 1...N_z^{\text{loc}} \tag{1.1}$$

while halo values are stored on the outer indices (i = 0 and i = $1...N_x^{\text{loc}} + 1$). This allows stencil operations to be applied uniformly across all interior points. Figures 1.4, 1.5 and 1.6 provides a visualisation local subdomain and indexing convention.

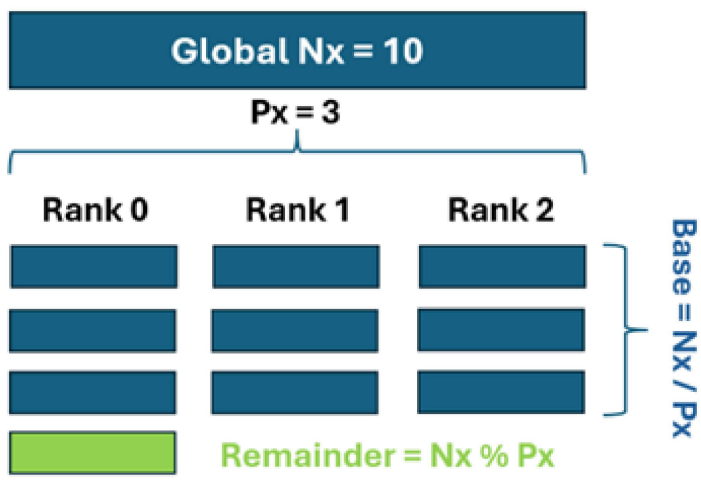


**Figure 1.3:** Non divisible block decomposition

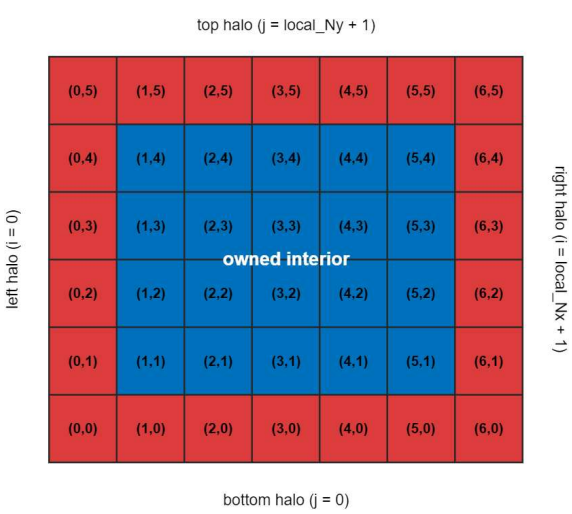


**Figure 1.4:** Local MPI subdomain with halo cells

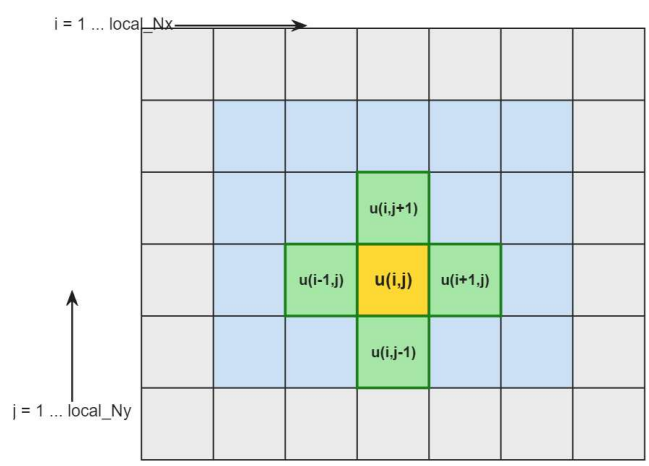


**Figure 1.5:** Halo based local indexing in 2D slice

Each rank sends its outermost interior planes to neighbouring ranks and receives corresponding data into its halo layers, as illustrated in Figure 1.6. Analogous exchanges are performed in the y and z direction

Initially, communication is implemeted using blocking MPI calls (MPI-Sendrecv). However, after analysis, non-blocking MPI calls were used. Specifically MPI-Irecv and MPI-Isend, followed by MPI-Waitall. This allows halo transfers to be initiated first, after which the strictly interior grid points are updated while communication is still in flight. Once all communication has completed, the boundary-adjacent points are updated using the newly received halo data. This partially overlaps communication with computation and reduces idle waiting compared with a fully blocking exchange. Since only boundary planes are communicated, the communication volume scales with the surface area of each subdomain rather than its volume.

Overall, the MPI workflow are as such:

1. Start iteration
2. Exchange x halos
3. Exchange y halos
4. Exchange z halos
5. Compute local Jacobi update
6. Swap u and u-new
7. Compute local residual vector
8. BLAS dnrm2 on local residual
9. MPI-Allreduce
10. Check convergence
11. Next iteration / stop

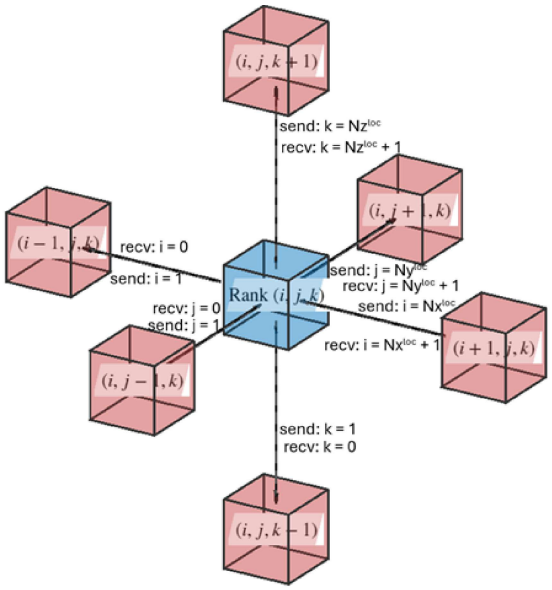


**Figure 1.6:** Indexing Convention

The residual norm is computed locally on each rank and combined into a global norm using MPI-Allreduce. After convergence, each rank packs its owned interior solution into a contiguous buffer and sends it to rank 0. Rank 0 then reconstructs the global solution using the known starting indices and local subdomain sizes, and writes the final result to "solution.txt".

# 2 Profiling and Benchmarking

Section-level timing using MPI-Wtime() showed that the most time-consuming component of the code is the residual evaluation, followed by halo exchange, with the Jacobi update contributing the least to the total runtime. In fact, the Jacobi update accounts for only a small fraction of the execution time compared to the residual and communication stages.

This indicates that the solver is not compute-bound, but instead dominated by communication and synchronisation overhead. The residual evaluation is particularly expensive as it requires an additional grid sweep, a BLAS norm computation (dnrm2), and a global reduction (MPI-Allreduce). Halo exchange represents the second largest cost due to repeated nearest-neighbour communication and associated memory operations.

As a result, optimisations targeting communication and residual evaluation are expected to yield the most significant performance improvements, whereas further optimisation of the Jacobi update would provide limited benefit. This behaviour is characteristic of memory- and communication-bound stencil solvers, where arithmetic intensity is low and performance is limited by data movement rather than floating-point operations.

Intel VTune hotspot analysis further confirmed that the dominant cost lies within solve-poisson-mpi, which contains the communication and residual evaluation loops. Among the subsidiary hotspots, the BLAS residual norm computation dnrm2, the global reduction MPI-Allreduce, and nearest-neighbour communication (MPI-Sendrecv in the original implementation) were all significant contributors as seen in Figure 2.1. Additional time was observed in memory movement routines, indicating that halo packing and unpacking operations also contribute non-negligibly to the total runtime.

One of the main inefficiencies identified was the repeated allocation and deallocation of halo communication buffers within each iteration. This behaviour is reflected in the VTune results through the presence of memmove and MPI communication routines as seen in Figure 2.1. To address this, the halo buffers were allocated once outside the iteration loop and reused throughout the computation, thereby eliminating unnecessary memory overhead.

Another optimisation targeted the repeated evaluation of the indexing function idx within the innermost loops. Previously, index values were recomputed multiple times per grid point, resulting in repeated integer arithmetic in performance-critical regions. This was improved by storing index values locally within the loop, reducing redundant computations and improving instruction efficiency.

Following these optimisations, both dnrm2 and MPI communication costs were reduced, as confirmed by VTune profiling

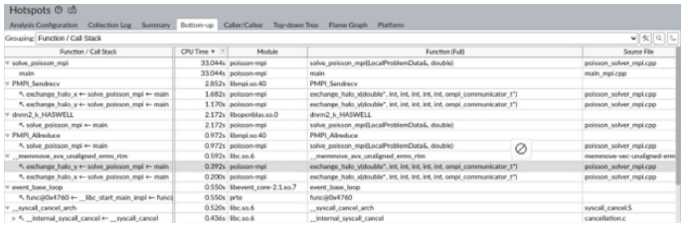

**Figure 2.1:** Intel VTune Pre-Optimisation

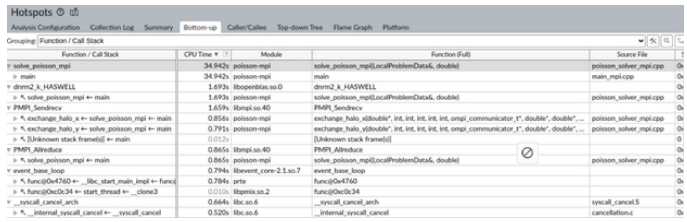

**Figure 2.2:** Intel VTune Post-Optimisation

observed in the time spent within communication and residual-related subroutines, particularly MPI-Allreduce and halo exchange routines. This demonstrates that the optimisations were effective in reducing overhead in these specific stages, even though the overall solver remains dominated by the Jacobi update and residual evaluation sweeps.

The timing results show that the optimisations lead to reduced execution time in most cases, particularly at higher process counts. This is reflected by increasingly negative percentage changes in runtime as the number of MPI ranks increases (Table 2.1). This trend indicates that the optimisations are most effective in communication-dominated regimes, where halo exchanges and global reductions constitute a larger fraction of the total runtime.

**Table 2.1:** Percentage change in section runtimes after optimisation for all benchmark cases. Percentage change was computed as $(T_{\text{new}} - T_{\text{old}})/T_{\text{old}} \times 100$, such that negative values indicate improved performance.

| Case | $N_x$ | $N_y$ | $N_z$ | $(P_x, P_y, P_z)$ | Ranks | Exchange (%) | Update (%) | Residual (%) |
|---|---|---|---|---|---|---|---|---|
| 1 | 16 | 16 | 16 | (1, 1, 1) | 1 | 5.47 | 0.87 | 24.35 |
| 2 | 24 | 24 | 24 | (1, 1, 1) | 1 | 22.10 | 2.42 | 1.16 |
| 3 | 32 | 32 | 32 | (1, 1, 1) | 1 | 25.25 | 13.80 | 0.16 |
| 4 | 16 | 16 | 16 | (2, 2, 1) | 4 | 48.99 | 3.25 | 28.22 |
| 5 | 16 | 16 | 16 | (2, 1, 2) | 4 | -41.96 | 2.56 | -47.59 |
| 6 | 24 | 24 | 24 | (1, 2, 2) | 4 | -0.33 | 16.33 | 4.43 |
| 7 | 32 | 32 | 32 | (2, 2, 1) | 4 | -4.04 | 15.89 | 8.61 |
| 8 | 33 | 31 | 29 | (2, 2, 1) | 4 | -0.93 | 2.86 | 3.84 |
| 9 | 24 | 24 | 24 | (2, 2, 2) | 8 | -0.89 | -13.32 | -12.93 |
| 10 | 24 | 24 | 24 | (4, 2, 1) | 8 | 1.33 | -10.66 | 1.89 |
| 11 | 32 | 32 | 32 | (2, 4, 1) | 8 | -3.16 | 10.13 | -5.48 |
| 12 | 40 | 40 | 40 | (2, 2, 2) | 8 | -6.89 | 0.21 | -4.86 |
| 13 | 41 | 39 | 37 | (2, 2, 2) | 8 | 0.18 | -5.69 | -5.86 |

Some cases exhibit slight increases in runtime following optimisation. These variations are attributed to factors such as load imbalance in non-divisible domain decompositions, cache effects, and measurement sensitivity for smaller problem sizes.

# 3 Parallel Efficiency and Scaling

The strong scaling results show that the optimised MPI solver performs well for the fixed $64^3$ problem. The number of MPI ranks increased from 1 to 48. Three test cases were considered. Test 1 and Test 2 represent larger runtime cases, while Test 3 is significantly smaller in magnitude and therefore a logarithmic scale was used in Figure 3.1.

For Test 1, the runtime decreased from 13.6826 s at 1 rank to 0.710006 s at 48 ranks. This corresponds to a speed-up of approximately 19.27 and a parallel efficiency of approximately 0.40 at 48 ranks. The results show that strong scaling is clearly beneficial for this case, although the scaling is not ideal. In particular, performance at 2 ranks is slightly worse than at 1 rank, indicating that communication and setup overhead dominate when only a small amount of parallel work is introduced. Beyond that point, the runtime decreases steadily with increasing rank count.

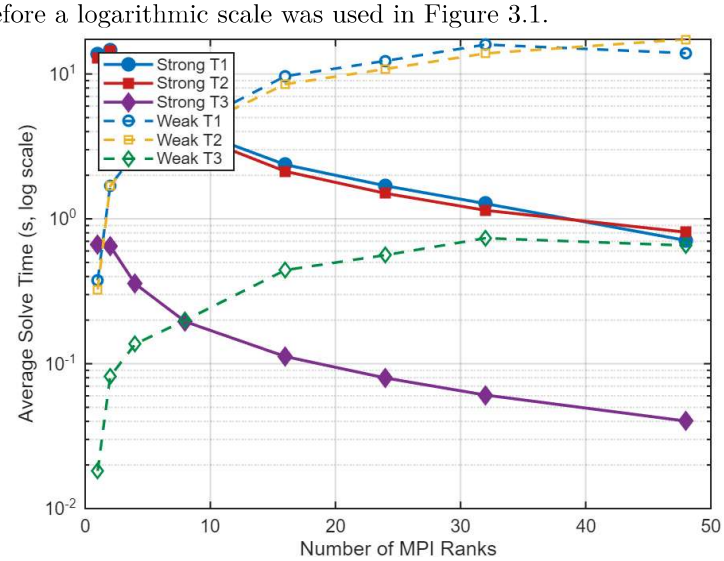


**Figure 3.1:** Strong Scaling Runtime

For Test 2, which corresponds to the original benchmark already used in the report, the runtime decreased from 12.9598 s at 1 rank to 0.8090 s at 48 ranks. This gives a speed-up of approximately 16.02 and a parallel efficiency of approximately 0.33 at 48 ranks. The overall behaviour is similar to Test 1: the code scales well over a moderate number of ranks, but efficiency drops progressively as the rank count increases. This is expected because halo communication, global residual reductions, and synchronization begin to represent a larger fraction of the total runtime

For Test 3, the runtime decreased from 0.662405 s at 1 rank to 0.0401885 s at 48 ranks. This corresponds to a speed-up of approximately 16.48 and an efficiency of approximately 0.34 at 48 ranks. Although the scaling trend is still good, this test is much smaller in absolute runtime, so the measurements are more sensitive to overheads.

Overall, the strong scaling results demonstrate that the MPI implementation achieves substantial reductions in runtime as the number of ranks increases. The full results are populated in Table 4.1 and Table 4.2. However, the efficiency decreases at higher rank counts because the amount of computation per process becomes smaller while communication and synchronization costs continue to grow. This behaviour is consistent with the distributed Jacobi method, where each iteration requires halo exchanges and a global reduction for the residual norm.

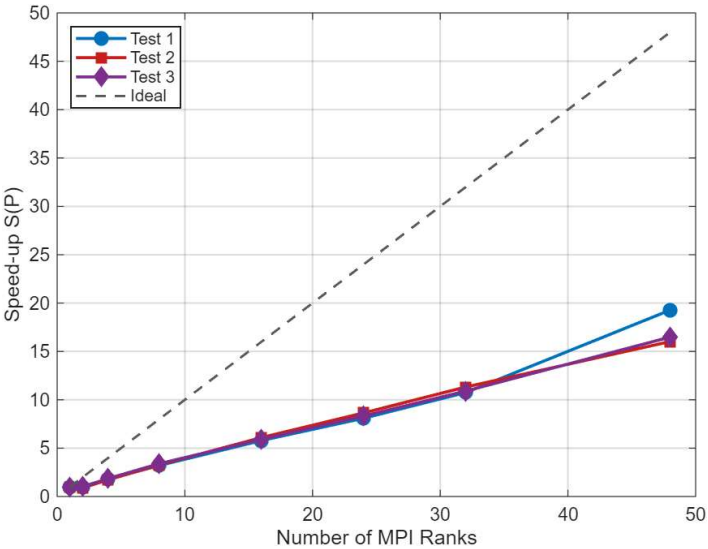


**Figure 3.2:** Strong Scaling Speedup

Weak scaling tests were performed by increasing the global problem size as the number of MPI ranks increased, so that the approximate amount of work per rank remained similar. Again, three test cases were examined and results plotted on Figure 3.3.

For Test 1, the runtime increased from 0.375781 s at 1 rank to 13.9238 s at 48 ranks. This gives a weak scaling efficiency of approximately 0.027 at 48 ranks. The ideal weak scaling result would show nearly constant runtime as the rank count increases, but this was not observed. Instead, the runtime rises significantly, indicating that communication and global synchronization costs increase rapidly with process count.

For Test 2, the original weak scaling benchmark, the runtime increased from 0.325558 s at 1 rank to 17.3387 s at 48 ranks. This corresponds to a weak scaling efficiency of approximately 0.0188 at 48 ranks. This confirms that the current implementation does not weak-scale well for large process counts. The main reason is that each iteration still requires communication of halo planes in all three coordinate directions together with a global residual reduction, so the communication overhead grows as the decomposition becomes finer.

For Test 3, the runtime increased from 0.018192 s at 1 rank to 0.654582 s at 48 ranks, giving a weak scaling efficiency of approximately 0.0278 at 48 ranks. Although the absolute runtime remains much smaller than in Tests 1 and 2, the same overall trend is observed: the runtime is not constant and instead grows steadily with increasing process count.

Taken together, the weak scaling results show that the MPI solver is more successful in strong scaling than in weak scaling. This is reasonable for a Jacobi-based stencil solver, because weak scaling still incurs increasing communication and synchronization overhead, particularly in the halo exchange and residual reduction steps. Consequently, while the solver benefits from additional ranks for fixed-size problems, it is less effective when the problem size is increased proportionally with the number of ranks.

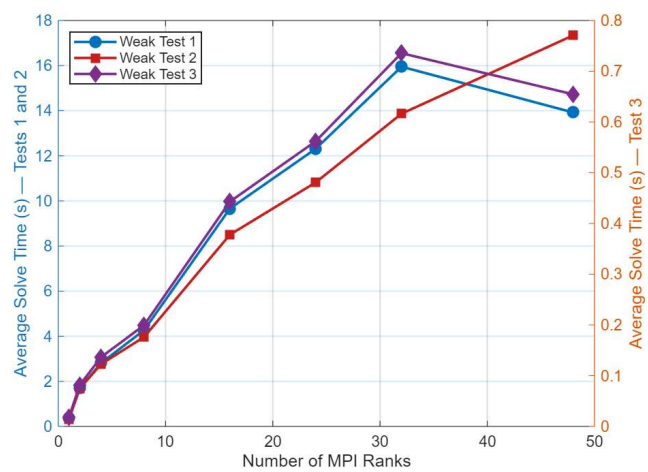


**Figure 3.3:** Weak Scaling Runtime

# 4 CUDA Programming

In addition to the MPI-only implementation, a hybrid MPI+CUDA version was tested. Three CUDA benchmarks were considered, corresponding to Test 1, Test 2, and Test 3. The purpose of these experiments was to determine whether offloading the Jacobi update and residual computation to the GPU improved performance relative to the MPI-only version.

A direct comparison between the MPI-only and hybrid MPI+CUDA implementations shows that CUDA does not consistently outperform MPI for the current implementation as seen in Figure 4.1.

For Test 1, the MPI runtime at 48 ranks was 0.710006 s, whereas the CUDA runtime was 58.972 s. For Test 2, the MPI runtime at 48 ranks was 0.8090 s, compared with 46.2264 s for CUDA. For Test 3, the MPI runtime at 48 ranks was 0.0401885 s, while the CUDA runtime was 52.9121 s. In all three cases, the CUDA implementation becomes substantially slower than MPI at high rank counts.

The converse is true for low rank counts. In some cases, CUDA offers an improvement relative to its own single-rank baseline, especially at 2, 4, or 8 ranks. This is seen as the speed-up largely follow the ideal linear growth initially but tapers off as the number of ranks increase in Figure 4.2. However, this does not necessarily mean that it outperforms the MPI-only implementation. The hybrid implementation still requires MPI halo exchange and global residual reduction on

result, the GPU acceleration is only useful when there is sufficient work per rank to offset those extra costs, which is not present in this study as the global grid size was $64^3$. Should the global grid size increase significantly to the point that each rank has a healthy amount of work, the effects of GPU offloading would be better seen.

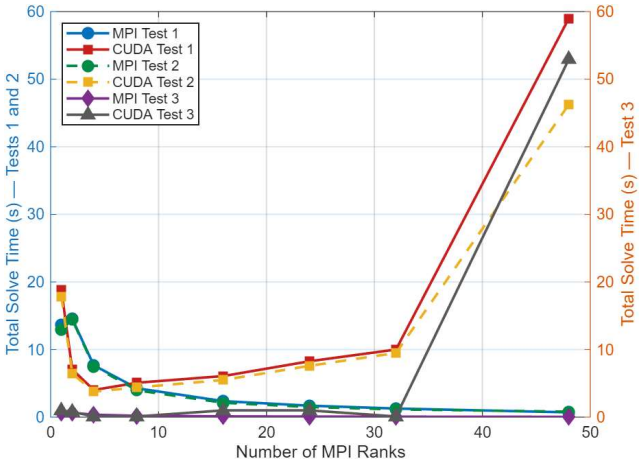


**Figure 4.1:** CUDA vs MPI speed-up comparison

The comparison therefore suggests that the current CUDA implementation is only advantageous, if at all, for relatively small numbers of MPI ranks. Once the local subdomain becomes too small, the GPU is underutilized and the communication and memory-transfer overheads dominate the total runtime.

The poor high-rank performance of the hybrid implementation can be explained by the structure of the code. In each iteration, halo exchange is still performed through MPI, and the updated data must be copied between host and device memory in order to launch the CUDA kernels and then make the updated solution available for communication. This means that the implementation does not avoid the core communication bottlenecks of the MPI solver, and in addition introduces extra transfer overhead.

At low rank counts, each MPI process owns a sufficiently large local subdomain that the GPU can still perform useful work. This explains the initial reduction in runtime seen in all three CUDA tests. However, as the number of ranks increases, the local domain shrinks, resulting in less work per kernel launch, poorer GPU occupancy, and a larger relative cost from communication and data movement. The severe slowdown observed at 48 ranks strongly suggests that the parallel decomposition has become too fine-grained for the hybrid approach to remain efficient.

Overall, the CUDA experiments show that simply offloading the stencil update to the GPU is not sufficient to guarantee better performance. In this implementation, the main limiting factors are host-device data transfer, MPI communication overhead, and the diminishing amount of local computation per rank. Therefore, the hybrid MPI+CUDA version demonstrates the feasibility of GPU acceleration, but does not outperform the MPI-only implementation in a robust or scalable way.

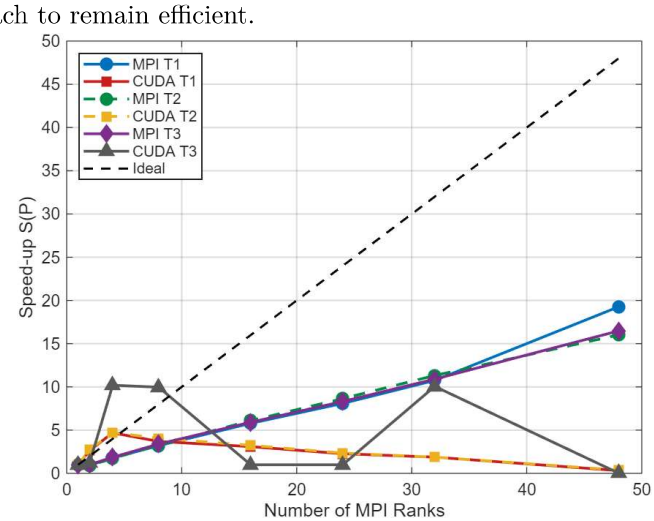


**Figure 4.2:** CUDA vs MPI speed-up comparison

**Table 4.1:** Strong scaling results for the optimised MPI solver across Tests 1, 2, and 3. Speed-up is defined as $S(P) = T_1/T_P$, and efficiency as $E(P) = S(P)/P$.

| **Ranks** | $(P_x, P_y, P_z)$ | **Test 1** | | | **Test 2** | | | **Test 3** | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | **Time (s)** | $S(P)$ | $E(P)$ | **Time (s)** | $S(P)$ | $E(P)$ | **Time (s)** | $S(P)$ | $E(P)$ |
| 1 | $(1, 1, 1)$ | 13.683 | 1.000 | 1.000 | 12.960 | 1.000 | 1.000 | 0.662 | 1.000 | 1.000 |
| 2 | $(2, 1, 1)$ | 14.559 | 0.940 | 0.470 | 14.435 | 0.898 | 0.449 | 0.650 | 1.020 | 0.510 |
| 4 | $(2, 2, 1)$ | 7.663 | 1.786 | 0.446 | 7.506 | 1.727 | 0.432 | 0.356 | 1.862 | 0.465 |
| 8 | $(2, 2, 2)$ | 4.269 | 3.205 | 0.401 | 3.967 | 3.267 | 0.408 | 0.195 | 3.395 | 0.424 |
| 16 | $(4, 2, 2)$ | 2.365 | 5.787 | 0.362 | 2.126 | 6.095 | 0.381 | 0.112 | 5.913 | 0.370 |
| 24 | $(4, 3, 2)$ | 1.691 | 8.093 | 0.337 | 1.499 | 8.644 | 0.360 | 0.080 | 8.313 | 0.346 |
| 32 | $(4, 4, 2)$ | 1.273 | 10.750 | 0.336 | 1.145 | 11.321 | 0.354 | 0.061 | 10.904 | 0.341 |
| 48 | $(4, 4, 3)$ | 0.710 | 19.271 | 0.401 | 0.809 | 16.020 | 0.334 | 0.040 | 16.482 | 0.343 |

**Table 4.2:** Weak scaling results for the optimised MPI solver across Tests 1, 2, and 3. Weak scaling efficiency is defined as $T_1/T_P$.

| **Ranks** | $(P_x, P_y, P_z)$ | **Test 1** | | | **Test 2** | | | **Test 3** | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | **Grid** | **Time (s)** | $T_1/T_P$ | **Grid** | **Time (s)** | $T_1/T_P$ | **Grid** | **Time (s)** | $T_1/T_P$ |
| 1 | $(1, 1, 1)$ | $32 \times 32 \times 32$ | 0.376 | 1.000 | $32 \times 32 \times 32$ | 0.326 | 1.000 | $32 \times 32 \times 32$ | 0.018 | 1.000 |
| 2 | $(2, 1, 1)$ | $64 \times 32 \times 32$ | 1.697 | 0.221 | $64 \times 32 \times 32$ | 1.684 | 0.193 | $64 \times 32 \times 32$ | 0.082 | 0.223 |
| 4 | $(2, 2, 1)$ | $64 \times 64 \times 32$ | 2.816 | 0.133 | $64 \times 64 \times 32$ | 2.768 | 0.118 | $64 \times 64 \times 32$ | 0.137 | 0.133 |
| 8 | $(2, 2, 2)$ | $64 \times 64 \times 64$ | 4.270 | 0.088 | $64 \times 64 \times 64$ | 3.965 | 0.082 | $64 \times 64 \times 64$ | 0.200 | 0.091 |
| 16 | $(4, 2, 2)$ | $128 \times 64 \times 64$ | 9.661 | 0.039 | $128 \times 64 \times 64$ | 8.503 | 0.038 | $128 \times 64 \times 64$ | 0.443 | 0.041 |
| 24 | $(4, 3, 2)$ | $128 \times 96 \times 64$ | 12.313 | 0.031 | $128 \times 96 \times 64$ | 10.820 | 0.030 | $128 \times 96 \times 64$ | 0.562 | 0.032 |
| 32 | $(4, 4, 2)$ | $128 \times 128 \times 64$ | 15.951 | 0.024 | $128 \times 128 \times 64$ | 13.867 | 0.023 | $128 \times 128 \times 64$ | 0.736 | 0.025 |